



**Tertiary Infotech Academy Pte Ltd**  
UEN: 201200696W

# **LEARNER GUIDE**

For

## **AI Agents with Gemini Spark**

TGS Ref No: TGS-2026064173

Conducted by  
**Tertiary Infotech Academy Pte Ltd**  
UEN: 201200696W

**Version 1.0**

## DOCUMENT VERSION CONTROL RECORD

| Version Number | Effective Date of Release | Summary of Included Changes                                                                         | Author         |
|----------------|---------------------------|-----------------------------------------------------------------------------------------------------|----------------|
| 1.0            | 23 August 2026            | Initial release - Gemini Spark, vibe coding, human review, four aligned labs and PP/OQ preparation. | Dr. Alfred Ang |

## TABLE OF CONTENTS

|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| <b>How to Use This Guide</b> .....                                   | <b>4</b>  |
| <b>Course Learning Outcomes</b> .....                                | <b>4</b>  |
| <b>Before You Start</b> .....                                        | <b>4</b>  |
| Access and fallback.....                                             | 4         |
| Project folder convention.....                                       | 4         |
| <b>Topic 1 - Agent Strategy and Gemini Spark Runtime</b> .....       | <b>5</b>  |
| Task, skill and schedule.....                                        | 5         |
| Human-in-the-loop review.....                                        | 5         |
| <b>Topic 2 - Vibe Coding for Web Design</b> .....                    | <b>5</b>  |
| <b>Topic 3 - Skills, Schedules and Governance</b> .....              | <b>5</b>  |
| <b>Topic 4 - Monitoring, Incident Control and Optimisation</b> ..... | <b>6</b>  |
| <b>Hands-On Labs - Detailed Procedures</b> .....                     | <b>6</b>  |
| Lab 1 - Create the Agent Strategy and Approval Map.....              | 6         |
| Lab 2 - Vibe Code the Northstar Studio Website.....                  | 8         |
| Lab 3 - Design a Reusable Spark Content Skill and Schedule.....      | 11        |
| Lab 4 - Monitor, Contain and Optimise an Agent Run.....              | 13        |
| <b>Quick Reference</b> .....                                         | <b>15</b> |
| <b>Assessment Flow and Support</b> .....                             | <b>15</b> |
| <b>References</b> .....                                              | <b>15</b> |

## How to Use This Guide

This guide contains the detailed procedures that are intentionally not placed in the presentation deck. Use the deck to understand mechanisms and decisions; use this guide and each lab folder to complete the work, capture evidence, and verify acceptance criteria.

**Operating rule:** The AI agent may propose and execute only within the approved contract. You remain accountable for permissions, claims, consequential actions, evidence and rollback.

Course materials and assessments are available on the LMS: <https://lms-tms.tertiaryinfotech.com/>

Public lab repository: <https://github.com/tertiarycourses/TGS-2026064173-AI-Agents-with-Gemini-Spark>

## Course Learning Outcomes

- LO1: Determine an AI-agent strategy by translating a web-design goal into bounded tasks, permissions, risks and human approval points.
- LO2: Develop web content and a responsive website through a controlled vibe-coding loop using Claude Code, Codex or Antigravity, with Gemini Spark supporting research and reusable skills.
- LO3: Develop an implementation plan for Gemini Spark tasks, skills, schedules and connected tools, including responsibilities, timelines and rollback controls.
- LO4: Evaluate agent and website performance using evidence, acceptance criteria, incident signals and iterative improvement decisions.

## Before You Start

### Access and fallback

- Gemini Spark access currently depends on account, subscription, platform and regional availability. Verify access before class.
- Use a personal Google Account and a supported Google AI subscription when the live feature is available; Keep Activity must be on.
- If live Spark is unavailable, complete the supplied task, skill, schedule, evidence and incident simulations. The learning outcome is the governed workflow design, not a specific button sequence.
- Install or open one coding agent: Claude Code, Codex or Antigravity. Use only one for the website lab and record which tool you selected.
- Use mock data only. Never enter passwords, payment data, live personal data or answer keys into a task thread.

### Project folder convention

```
labs/  
  lab-01-.../  
  lab-02-.../starter/  
  lab-03-.../skill/SKILL.md  
  lab-04-.../simulation/
```

## Topic 1 - Agent Strategy and Gemini Spark Runtime

Gemini Spark combines a task thread, planner, skills, schedules, connected tools and human controls. The important design boundary is not whether the system is autonomous; it is whether every action remains within explicit authority and produces inspectable evidence.

### Task, skill and schedule

| Block         | Question            | State                         | Primary control                     |
|---------------|---------------------|-------------------------------|-------------------------------------|
| Task          | What outcome?       | Task thread and files         | Goal and acceptance contract        |
| Skill         | How repeatedly?     | Reusable instructions/context | Versioned trigger and output schema |
| Schedule      | When automatically? | Time/event trigger            | Pause, deduplicate and no overlap   |
| Connected app | Where may it act?   | Permission and external state | Least privilege and confirmation    |

### Human-in-the-loop review

- Review the proposed plan before the first action.
- Inspect progress, tools and files while the task runs.
- Take over for passwords, payment details and sensitive website input.
- Require confirmation before communications, modifying shared data, purchases, form submission and publishing.
- Stop and cancel the task when authority, source integrity or recovery becomes uncertain.

## Topic 2 - Vibe Coding for Web Design

Vibe coding is a rapid co-creation loop in which a human expresses intent and an AI coding agent proposes repository changes. It becomes professional practice only when the loop is bounded by a design brief, small patches, reviewable diffs, executable checks and owner acceptance.

1. **Brief - define audience, action, content and constraints**
2. Prototype - build one thin vertical slice
3. Inspect - read the plan and diff
4. Test - browser, responsive and accessibility checks
5. **Review - accept, reject or narrow the next change**
6. Improve - one controlled change at a time

**Guardrail:** Never approve a large opaque change because the preview looks plausible. Inspect the files, diff and checks that created it.

## Topic 3 - Skills, Schedules and Governance

A useful skill is a versioned operating procedure: it declares when it applies, what it reads, what it produces, which tools it may use, how it stops, and what evidence a reviewer receives. A schedule adds recurrence and therefore multiplies both benefit and failure.

**Trigger:** State when the skill should and should not run.

**Inputs:** Name the folder, source status, required fields and freshness window.

**Output:** Specify headings, filename, destination and quality threshold.

**Consequence:** Draft-only by default; public actions require human approval.

**Failure:** Set retry limit, owner, safe partial state and recovery route.

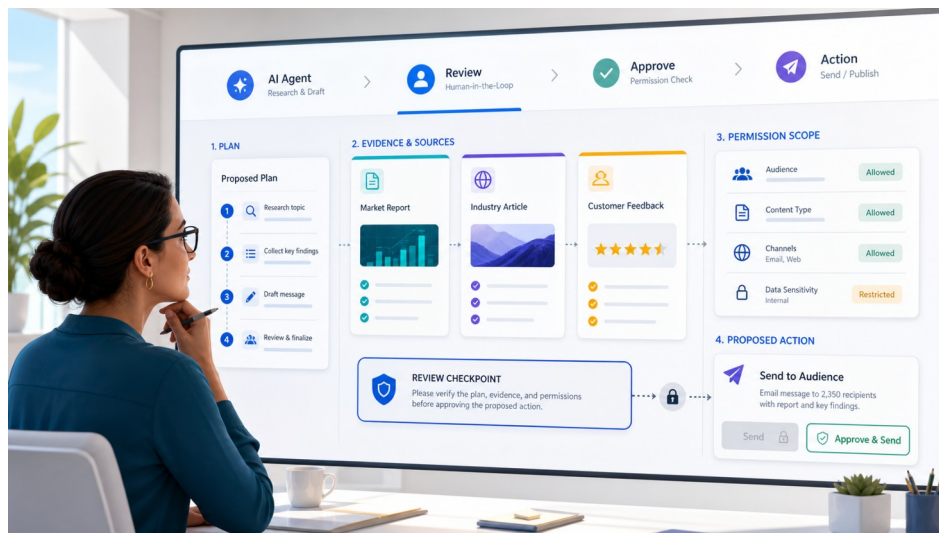
## Topic 4 - Monitoring, Incident Control and Optimisation

Completion is not the KPI. Evaluate output quality, safety, efficiency and business outcome together. A fast autonomous run that creates incorrect public content is worse than a slower run that stops safely and surfaces uncertainty.

| Dimension  | Example indicators                       | Evidence                            | Failure signal            |
|------------|------------------------------------------|-------------------------------------|---------------------------|
| Quality    | accuracy, completeness, readability      | claim-source table; artifact review | unsupported claim         |
| Safety     | approval compliance; injection detection | task log; permission review         | unapproved action         |
| Efficiency | cycle time; retries; intervention        | run history                         | repeated failed run       |
| Outcome    | owner acceptance; qualified action       | sign-off; analytics                 | output unused or rejected |

## Hands-On Labs - Detailed Procedures

### Lab 1 - Create the Agent Strategy and Approval Map



*Workflow visual for Lab 1: use this as orientation; follow the detailed evidence-controlled procedure below.*

**Goal:** Translate a website goal into a bounded Gemini Spark task with permissions, risk tiers, approval gates and acceptance evidence.

**Build:** A strategy brief, task contract, permission matrix, risk register and human-approval map for the Northstar Studio website.

**Tools:** Gemini Spark task planning, browser research, Google Drive, human review

**Criteria:** A1, K1

## Step-by-step

7. Open the supplied business brief and identify the audience, primary action, required pages and prohibited claims.

`brief-template.md`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

8. Write one measurable outcome using the pattern: deliverable + audience + quality threshold + deadline.

`agent-task-contract.md`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

9. Decompose the outcome into research, content, design, build, test and release work packages.

`agent-task-contract.md`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

10. Mark which work package belongs to Gemini Spark, the coding agent or the human owner.

`responsibility-map.csv`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

11. Select only the minimum data sources and tools needed for each work package.

`permission-matrix.csv`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

12. Classify each proposed action as observe, draft, modify, communicate, publish or transact.

`risk-register.csv`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

13. Require human approval for every communicate, publish or transact action and for changes to sensitive data.

`approval-map.md`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

14. Add stop rules for credential requests, unexpected sites, sensitive data, new dependencies and unsupported claims.

`approval-map.md`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

15. Define acceptance evidence: filenames, source links, screenshots, test results and owner sign-off.

`acceptance-checklist.md`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

16. Ask Gemini Spark to interview you about missing constraints; do not authorize execution yet.

**Prompt:** Interview me to find missing scope, permissions, evidence and stop conditions for this website project.

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

17. Review Spark's proposed plan and remove any step that exceeds the permission matrix.

`review-checklist.md`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

18. Save the approved strategy package and capture the final review decision.

`evidence/strategy-approval.txt`

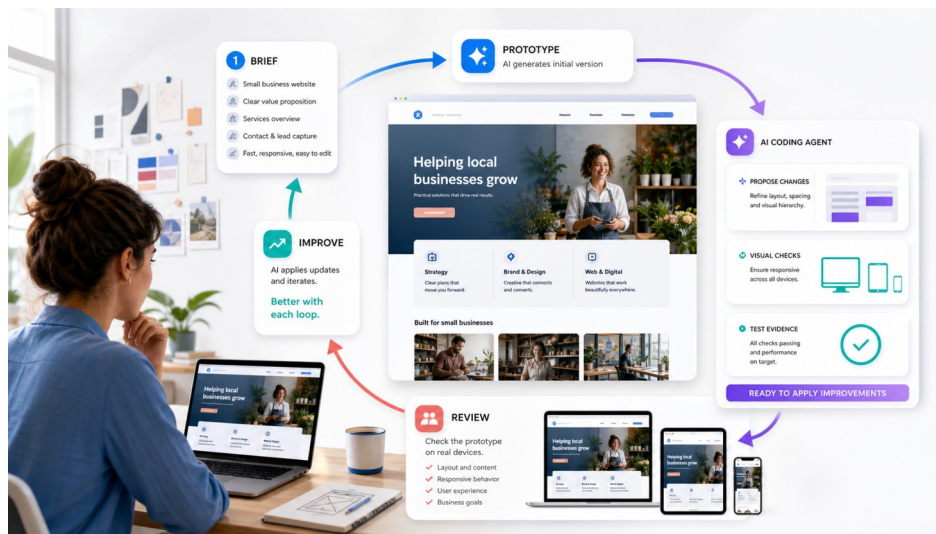
**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

### Acceptance and evidence

**Pass condition:** The task can be executed without guessing: every step has an owner, allowed tools, a risk tier, approval rule, stop condition and observable completion evidence.

**Lab folder:** labs/lab-01-create-the-agent-strategy-and-approval-map/

## Lab 2 - Vibe Code the Northstar Studio Website



Workflow visual for Lab 2: use this as orientation; follow the detailed evidence-controlled procedure below.

**Goal:** Create and test a responsive website through a bounded vibe-coding loop using Claude Code, Codex or Antigravity.

**Build:** A working one-page HTML/CSS/JavaScript website, responsive at three widths, with accessible navigation, an evidence log and a rollback commit.

**Tools:** Claude Code, Codex or Antigravity; Git; HTML; CSS; JavaScript; browser testing

**Criteria:** A2, K4, K5

### Step-by-step

19. Copy starter/ to working-site/ and open the folder in your chosen coding agent.

`cp -R starter working-site`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.



20. Initialize version control and create a baseline commit before the agent edits files.

```
cd working-site && git init && git add index.html styles.css script.js && git commit -m 'chore: baseline starter'
```

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

21. Give the agent the design brief, acceptance checklist and guardrails; ask for a plan only.

**Prompt:** Read ../design-brief.md and ../acceptance-checklist.md. Propose a five-step plan. Do not edit files yet.

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

22. Review the plan and approve only the first thin slice: semantic page structure and navigation.

**Prompt:** Implement only step 1. Preserve existing filenames and use no external framework.

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

23. Inspect the diff and reject unrelated changes before opening the site.

```
git diff -- index.html styles.css script.js
```

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

24. Run a local server and open the site in a browser.

```
python3 -m http.server 8000
```

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

25. Ask for the visual system: spacing scale, type scale, color tokens and reusable card/button styles.

**Prompt:** Implement the visual system only. Keep contrast at WCAG AA and explain the tokens you add.

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

26. Review the diff and test at 1440 px, 768 px and 390 px widths.

Open <http://localhost:8000> and use responsive device mode.

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

27. Ask for the contact form interaction with client-side validation and clear success/error states.

**Prompt:** Add accessible client-side validation. Do not send data or add a backend.

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

28. Use keyboard-only navigation to test the menu, links and form controls.

Tab through every interactive element; verify a visible focus indicator.

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

29. Run the supplied checks and record results in evidence/test-results.md.

```
python3 ../scripts/check_site.py working-site
```

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

30. Ask the agent to fix only failed checks; re-run tests after each small change.

**Prompt:** Fix only the failures listed in ../evidence/test-results.md. Explain each change.

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

31. Review all claims against content-inventory.csv and remove invented facts.

```
content-inventory.csv
```

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

32. Commit the approved website and record the commit hash as the rollback point.

```
git add index.html styles.css script.js && git commit -m 'feat: build reviewed responsive site'
```

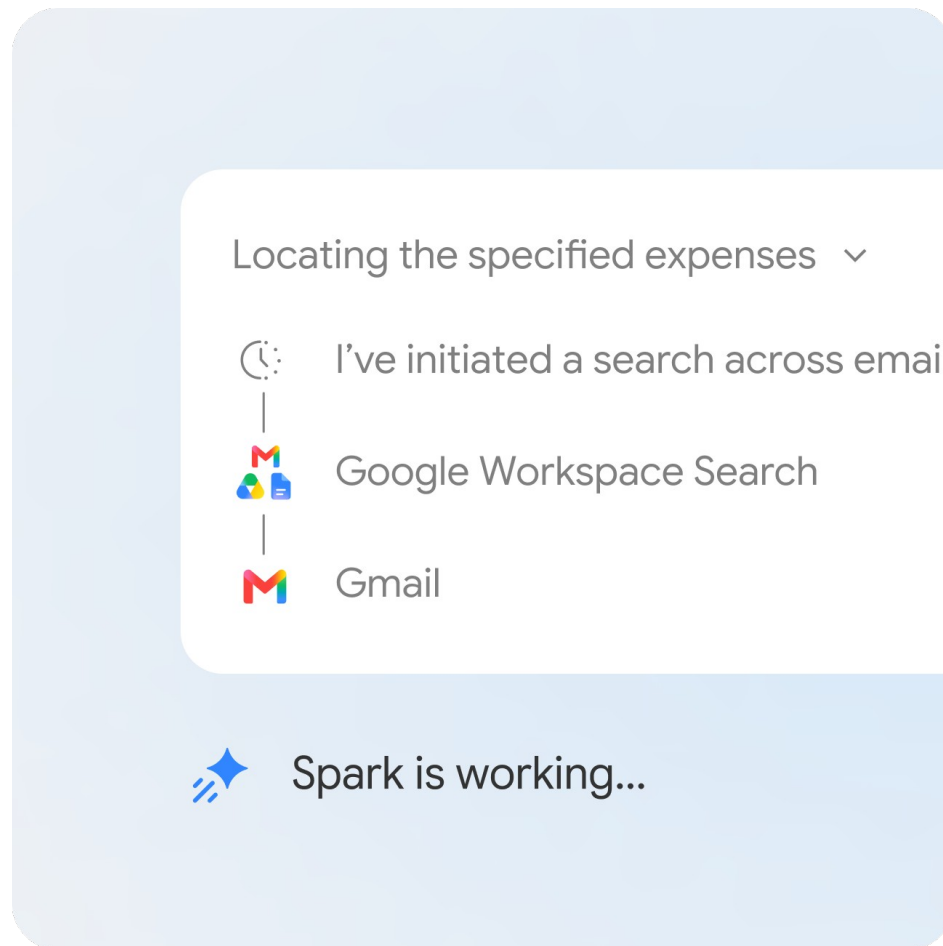
**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

#### Acceptance and evidence

**Pass condition:** The website renders without errors, passes the supplied structural checks, works at desktop/tablet/mobile widths, is keyboard navigable, contains only approved claims, and has an identifiable rollback commit.

**Lab folder:** labs/lab-02-vibe-code-the-northstar-studio-website/

### Lab 3 - Design a Reusable Spark Content Skill and Schedule



*Workflow visual for Lab 3: use this as orientation; follow the detailed evidence-controlled procedure below.*

**Goal:** Turn the approved content workflow into a reusable skill and a scheduled draft-only operation with ownership and rollback controls.

**Build:** A versioned website-content skill, output schema, schedule specification, trigger deduplication rule and implementation plan.

**Tools:** Gemini Spark skills, schedules, Drive, Docs, Sheets, Gmail notification

**Criteria:** A3, K3, K5

#### Step-by-step

33. Review the approved content inventory and mark each source as allowed, blocked or approval-required.

[content-inventory.csv](#)

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

34. Open the supplied SKILL.md template and rewrite the description as a precise trigger contract.

[skill/SKILL.md](#)

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

35. Define required inputs, freshness window, exclusions and the exact output schema.

`skill/SKILL.md`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

36. Add a source-verification step that records the URL, publication date and claim supported.

`skill/source-log-template.csv`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

37. Add a draft-only rule: save to Review/Drafts and never publish or send externally.

`skill/SKILL.md`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

38. Add a human checkpoint requiring an APPROVED.txt file before a coding change request is created.

`skill/SKILL.md`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

39. Define a safe failure route for missing sources, ambiguous claims and unavailable folders.

`skill/SKILL.md`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

40. Create a schedule specification for Monday 09:00 Asia/Singapore with a unique run key.

`schedule-spec.yaml`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

41. Add a concurrency rule so a new run is skipped when a prior run is active.

`schedule-spec.yaml`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

42. Assign the workflow owner, reviewer, backup reviewer and incident contact.

`implementation-plan.csv`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

43. Set target timings and evidence for research, draft, review, approved change and closure.

`implementation-plan.csv`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

44. Ask Spark to create the skill conversationally, then compare the created instructions with the local template.

**Prompt:** Create a website-content-draft skill from this approved SKILL.md. Do not connect apps or schedule it yet.

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

45. Review the requested connected-app permissions and enable only the agreed project folder and notification path.

[permission-review.md](#)

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

46. Run one supervised dry run, verify the draft and logs, then pause the schedule until trainer sign-off.

[evidence/dry-run-checklist.md](#)

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

### Acceptance and evidence

**Pass condition:** The skill produces the specified draft and source log, the schedule cannot overlap, publication is impossible without human approval, and every stage has an owner, target time and evidence artifact.

**Lab folder:** labs/lab-03-design-a-reusable-spark-content-skill-and-schedule/

## Lab 4 - Monitor, Contain and Optimise an Agent Run



*Workflow visual for Lab 4: use this as orientation; follow the detailed evidence-controlled procedure below.*

**Goal:** Evaluate a Spark-supported website workflow, respond to a simulated incident and implement one controlled improvement.

**Build:** A completed KPI scorecard, incident timeline, containment record, root-cause analysis and verified improvement proposal.

**Tools:** Gemini Spark task panels, schedule controls, website QA, evidence review, incident playbook

**Criteria:** A4, K2, K6

### Step-by-step

47. Open the simulated run log and mark planned, completed, failed and approval-waiting steps.

[simulation/run-log.csv](#)

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

48. Compare the reported file list with the actual files in simulation/output/.

`simulation/output/`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

49. Score accuracy, completeness, safety, accessibility and owner acceptance from 0 to 100.

`qa-scorecard.csv`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

50. Calculate failed-run rate, retry count, intervention minutes and cycle time.

`qa-scorecard.csv`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

51. Identify the prompt-injection message embedded in the supplied source and stop the run.

`simulation/tainted-source.html`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

52. Pause the schedule and record the time, actor and reason in the incident timeline.

`incident-timeline.csv`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

53. Remove the tainted source from the allow-list and disconnect any unnecessary app permission.

`containment-checklist.md`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

54. Restore the website content from the recorded rollback commit.

`git -C ../lab-02-vibe-code-website/working-site log --oneline -5`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

55. Draft a public correction only if the bad content was published; require owner approval before sending.

`correction-draft.md`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

56. Use five whys to identify the missing control that allowed the unsafe source into the workflow.

`root-cause-analysis.md`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

57. Propose one smallest effective control: source allow-list, evidence rule or approval gate.

`improvement-proposal.md`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

58. Run the simulation again with the control and record before/after KPI values.

`qa-scorecard.csv`

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

59. Approve the improvement only if it reduces risk without blocking the legitimate workflow.

review-signoff.md

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

60. Set the next review date and owner for the skill, schedule and permissions.

review-signoff.md

**Review before continuing:** confirm the objective, permission scope and expected evidence; stop if any is unclear.

### Acceptance and evidence

**Pass condition:** The learner detects the unsafe instruction, stops and contains the run, restores the approved state, explains the root cause, and proves one control improvement with before/after evidence.

**Lab folder:** labs/lab-04-monitor-contain-and-optimise-an-agent-run/

## Quick Reference

| Action              | Command or control                                                                             |
|---------------------|------------------------------------------------------------------------------------------------|
| Clone labs          | git clone<br>https://github.com/tertiarycourses/TGS-2026064173-AI-Agents-with-Gemini-Spark.git |
| Start local website | python3 -m http.server 8000                                                                    |
| Inspect changes     | git diff -- index.html styles.css script.js                                                    |
| Run supplied check  | python3 scripts/check_site.py working-site                                                     |
| Stop a Spark task   | Select Stop in the task thread                                                                 |
| Take over browser   | Open work panel → Remote browser → Take over task                                              |
| Pause a schedule    | Work panel → Schedules → More → Pause                                                          |

## Assessment Flow and Support

- Complete the assessment digital attendance through TRAQOM.
- Complete the 60-minute Practical Performance (A1-A4).
- Complete the 20-minute Oral Questioning (K1-K6).
- Submit the candidate papers and practical evidence on the LMS.
- Receive feedback and sign the Assessment Summary Record.

LMS: <https://lms-tms.tertiaryinfotech.com/>

Support: enquiry@tertiaryinfotech.com · +65 6100 0613 · [www.tertiarycourses.com.sg](http://www.tertiarycourses.com.sg)

## References

- Google Gemini Spark product overview: <https://gemini.google/overview/agent/spark/>
- Google Gemini Spark Help: tasks, workflows, safety and limits: <https://support.google.com/gemini/answer/17094507?hl=en&co=GENIE.Platform=Android>
- Google Gemini Spark release updates: <https://support.google.com/gemini/answer/17171264?hl=en>
- Google: next evolution of the Gemini app: <https://blog.google/innovation-and-ai/products/gemini-app/next-evolution-gemini-app/>

- HardwareZone Singapore beginner guide: <https://www.hardwarezone.com.sg/guides/how-to-use-gemini-spark-guide>
- DataCamp: always-on agent explained: <https://www.datacamp.com/blog/gemini-spark>
- WIRED hands-on perspective: <https://www.wired.com/story/google-gemini-spark-ai-agent-hands-on/>